
1. Why This Topic Is Important

Many students can write basic SQL queries, but they get confused when queries become slightly complex.

Common confusion:

Why WHERE cannot use aggregate functions?

Why HAVING is used after GROUP BY?

Why SELECT alias does not work in WHERE?

Why ORDER BY can use alias?

Why LEFT JOIN behaves like INNER JOIN sometimes?

The answer to all these questions is:

SQL logical execution order

SQL is written in one order, but logically processed in a different order.

2. SQL Written Order

Normally, we write SQL like this:

```
SELECT
FROM
JOIN
WHERE
GROUP BY
HAVING
ORDER BY
LIMIT
```

Example:

```
SELECT
  customer_id,
```

```
SUM(order_amount) AS total_amount  
FROM orders  
WHERE order_status = 'Delivered'  
GROUP BY customer_id  
HAVING SUM(order_amount) > 50000  
ORDER BY total_amount DESC;
```

But SQL does not logically execute from **SELECT**.

3. SQL Logical Execution Order

SQL logically executes in this order:

1. FROM
2. JOIN / ON
3. WHERE
4. GROUP BY
5. HAVING
6. SELECT
7. DISTINCT
8. ORDER BY
9. LIMIT

Easy way to remember:

First SQL finds the data.
Then it joins the data.
Then it filters rows.
Then it groups the data.
Then it filters groups.
Then it selects columns.
Then it removes duplicates.
Then it sorts.
Then it limits the output.

4. Complete Practice Setup

Create Database

```
DROP DATABASE IF EXISTS sql_execution_order;
```

```
CREATE DATABASE sql_execution_order;
```

```
USE sql_execution_order;
```

Create customers table

```
CREATE TABLE customers (  
    customer_id INT PRIMARY KEY,  
    customer_name VARCHAR(50),  
    city VARCHAR(50)  
);
```

Create orders table

```
CREATE TABLE orders (  
    order_id INT PRIMARY KEY,  
    customer_id INT,  
    product_category VARCHAR(50),  
    order_amount DECIMAL(10,2),  
    order_status VARCHAR(30),  
    order_date DATE  
);
```

Insert customers

```
INSERT INTO customers  
(customer_id, customer_name, city)  
VALUES  
(1, 'Anurag', 'Bengaluru'),  
(2, 'Stuti', 'Pune'),  
(3, 'Rahul', 'Delhi'),  
(4, 'Priya', 'Mumbai'),  
(5, 'Neha', 'Bengaluru');
```

Insert orders

```
INSERT INTO orders
(order_id, customer_id, product_category, order_amount, order_status, order_date)
VALUES
(101, 1, 'Electronics', 50000.00, 'Delivered', '2026-06-01'),
(102, 1, 'Accessories', 1000.00, 'Delivered', '2026-06-02'),
(103, 2, 'Accessories', 2500.00, 'Delivered', '2026-06-03'),
(104, 2, 'Electronics', 48000.00, 'Delivered', '2026-06-04'),
(105, 3, 'Accessories', 3500.00, 'Cancelled', '2026-06-05'),
(106, 4, 'Electronics', 15000.00, 'Pending', '2026-06-06'),
(107, 5, 'Accessories', 1200.00, 'Delivered', '2026-06-07'),
(108, 5, 'Electronics', 22000.00, 'Delivered', '2026-06-08');
```

Check the data

```
SELECT * FROM customers;
```

```
SELECT * FROM orders;
```

5. Main Example Query

Scenario

Business asks:

Find cities where delivered order revenue is more than 50,000.
Show the city and total revenue.
Sort highest revenue first.

Query

```
SELECT
  c.city,
  SUM(o.order_amount) AS total_revenue
```

```
FROM customers c
INNER JOIN orders o
  ON c.customer_id = o.customer_id
WHERE o.order_status = 'Delivered'
GROUP BY c.city
HAVING SUM(o.order_amount) > 50000
ORDER BY total_revenue DESC;
```

Now let us understand how SQL logically processes this query.

6. Step-by-Step Execution

Step 1: FROM

SQL first identifies the base table.

```
FROM customers c
```

Meaning:

Start from the customers table.

At this point, SQL knows customer details like:

```
customer_id
customer_name
city
```

Step 2: JOIN / ON

Then SQL joins customers with orders.

```
INNER JOIN orders o
  ON c.customer_id = o.customer_id
```

Meaning:

Match customers with their orders using `customer_id`.

Why this step is needed?

City is available in the `customers` table.

Order amount is available in the `orders` table.

So we need a join.

Important point:

JOIN happens before WHERE.

Step 3: WHERE

After joining, SQL filters row-level data.

WHERE `o.order_status = 'Delivered'`

Meaning:

Keep only delivered orders.

Remove cancelled and pending orders.

Important point:

WHERE filters individual rows before grouping.

So first delivered rows are selected.

Then aggregation happens.

Step 4: GROUP BY

Now SQL groups data.

GROUP BY c.city

Meaning:

Create city-level groups.

Example:

Bengaluru → orders from Bengaluru customers

Pune → orders from Pune customers

Delhi → orders from Delhi customers

Mumbai → orders from Mumbai customers

Step 5: HAVING

After grouping, SQL filters grouped results.

HAVING SUM(o.order_amount) > 50000

Meaning:

Keep only those cities where total delivered revenue is more than 50,000.

Important point:

HAVING filters groups.

WHERE filters rows.

Step 6: SELECT

Now SQL selects the final columns to display.

SELECT

c.city,

SUM(o.order_amount) AS total_revenue

Meaning:

Show city and total revenue.

Important point:

SELECT is logically processed after WHERE, GROUP BY, and HAVING.

This is why SELECT alias is not available in WHERE.

Step 7: ORDER BY

Finally, SQL sorts the output.

ORDER BY total_revenue DESC

Meaning:

Sort cities by highest revenue first.

Important point:

ORDER BY can use SELECT alias because ORDER BY runs after SELECT.

7. WHERE vs HAVING

This is one of the most important concepts.

WHERE

WHERE filters rows before grouping.

Example:

WHERE order_status = 'Delivered'

This means:

Keep only delivered order rows before aggregation.

HAVING

HAVING filters groups after aggregation.

Example:

```
HAVING SUM(order_amount) > 50000
```

This means:

After calculating total revenue, keep only groups where revenue is more than 50,000.

Simple Difference

WHERE = filters rows

HAVING = filters groups

8. Common Mistake 1: Using Aggregate Function in WHERE

Wrong Query

```
SELECT
  customer_id,
  SUM(order_amount) AS total_amount
FROM orders
WHERE SUM(order_amount) > 50000
GROUP BY customer_id;
```

Why this is wrong?

WHERE executes before **GROUP BY**.

At the time of **WHERE**, **SUM(order_amount)** does not exist yet.

Correct Query

```
SELECT
  customer_id,
  SUM(order_amount) AS total_amount
FROM orders
GROUP BY customer_id
HAVING SUM(order_amount) > 50000;
```

Remember

Aggregate filters should go in **HAVING**, not **WHERE**.

9. Common Mistake 2: Using SELECT Alias in WHERE

Wrong Query

```
SELECT
  order_amount AS amount
FROM orders
WHERE amount > 10000;
```

Why this is wrong?

WHERE executes before **SELECT**.

The alias **amount** is created in **SELECT**.

So **WHERE** does not know **amount**.

Correct Query

```
SELECT
    order_amount AS amount
FROM orders
WHERE order_amount > 10000;
```

Remember

SELECT alias is not available in WHERE.

10. Why ORDER BY Can Use Alias

This query works:

```
SELECT
    customer_id,
    SUM(order_amount) AS total_amount
FROM orders
GROUP BY customer_id
ORDER BY total_amount DESC;
```

Why?

Because **ORDER BY** executes after **SELECT**.

So alias **total_amount** is already created.

11. Common Mistake 3: LEFT JOIN with WHERE Filter

Scenario

Business asks:

Show all customers and their delivered orders if available.

Wrong Query

```
SELECT
  c.customer_name,
  o.order_id,
  o.order_status
FROM customers c
LEFT JOIN orders o
  ON c.customer_id = o.customer_id
WHERE o.order_status = 'Delivered';
```

Why this can be wrong?

`LEFT JOIN` keeps all customers.

But `WHERE o.order_status = 'Delivered'` removes rows where order details are NULL.

So customers with no delivered orders disappear.

This makes LEFT JOIN behave like INNER JOIN.

Correct Query

```
SELECT
  c.customer_name,
  o.order_id,
  o.order_status
FROM customers c
LEFT JOIN orders o
  ON c.customer_id = o.customer_id
  AND o.order_status = 'Delivered';
```

Remember

In LEFT JOIN, if you want to preserve all left table rows, put right table filters inside ON condition.

12. Practice Questions with Solutions

Question 1

Find total revenue by customer.

Scenario

Business wants to know how much each customer has ordered.

Solution

```
SELECT
  customer_id,
  SUM(order_amount) AS total_revenue
FROM orders
GROUP BY customer_id;
```

Why this query?

We group orders by customer and calculate total amount using **SUM**.

Question 2

Find total delivered revenue by customer.

Scenario

Business wants revenue only from delivered orders.

Solution

```
SELECT
  customer_id,
  SUM(order_amount) AS total_delivered_revenue
FROM orders
WHERE order_status = 'Delivered'
GROUP BY customer_id;
```

Why this query?

First filter delivered orders using WHERE.
Then group by customer.

Question 3

Find customers whose delivered revenue is more than 50,000.

Scenario

Business wants high-value customers based only on delivered orders.

Solution

```
SELECT
  customer_id,
  SUM(order_amount) AS total_delivered_revenue
FROM orders
WHERE order_status = 'Delivered'
GROUP BY customer_id
HAVING SUM(order_amount) > 50000;
```

Why this query?

WHERE filters delivered rows.

GROUP BY groups by customer.

HAVING filters customers whose total delivered revenue is more than 50,000.

Question 4

Find city-wise delivered revenue.

Scenario

Business wants to compare delivered revenue by city.

Solution

```
SELECT
  c.city,
  SUM(o.order_amount) AS total_revenue
FROM customers c
INNER JOIN orders o
  ON c.customer_id = o.customer_id
WHERE o.order_status = 'Delivered'
GROUP BY c.city;
```

Why this query?

City is in **customers**.

Order amount is in **orders**.

So first join both tables.

Then filter delivered orders.

Then group by city.

Question 5

Find cities where delivered revenue is more than 50,000.

Scenario

Business wants only high-performing cities.

Solution

```
SELECT
  c.city,
  SUM(o.order_amount) AS total_revenue
FROM customers c
INNER JOIN orders o
  ON c.customer_id = o.customer_id
WHERE o.order_status = 'Delivered'
GROUP BY c.city
HAVING SUM(o.order_amount) > 50000;
```

Why this query?

Revenue by city is an aggregate result.

So we cannot filter it using WHERE.

We need HAVING.

Question 6

Sort cities by delivered revenue from highest to lowest.

Scenario

Business wants to see top cities first.

Solution

```
SELECT
  c.city,
  SUM(o.order_amount) AS total_revenue
FROM customers c
INNER JOIN orders o
  ON c.customer_id = o.customer_id
WHERE o.order_status = 'Delivered'
GROUP BY c.city
ORDER BY total_revenue DESC;
```

Why this query?

ORDER BY runs after SELECT.

So it can use alias **total_revenue**.

Question 7

Find delivered revenue by product category.

Scenario

Business wants to know which product category is generating revenue.

Solution

```
SELECT
    product_category,
    SUM(order_amount) AS total_revenue
FROM orders
WHERE order_status = 'Delivered'
GROUP BY product_category;
```

Why this query?

First keep only delivered orders.
Then group by product category.
Then calculate revenue.

Question 8

Find product categories where delivered revenue is more than 50,000.

Scenario

Business wants only strong-performing categories.

Solution

```
SELECT
    product_category,
    SUM(order_amount) AS total_revenue
FROM orders
WHERE order_status = 'Delivered'
GROUP BY product_category
HAVING SUM(order_amount) > 50000;
```

Why this query?

WHERE filters rows.

HAVING filters grouped category revenue.

Question 9

Find top 2 customers by delivered revenue.

Scenario

Business wants the top 2 customers by delivered revenue.

Solution

```
SELECT
    customer_id,
    SUM(order_amount) AS total_delivered_revenue
FROM orders
WHERE order_status = 'Delivered'
GROUP BY customer_id
ORDER BY total_delivered_revenue DESC
LIMIT 2;
```

Why this query?

Execution flow:

```
FROM orders
WHERE delivered
GROUP BY customer_id
SELECT total revenue
ORDER BY revenue descending
LIMIT 2
```

Question 10

Show all customers and their delivered orders if available.

Scenario

Business wants all customers, even if they do not have delivered orders.

Solution

```
SELECT
    c.customer_name,
```

```
c.city,  
o.order_id,  
o.order_status  
FROM customers c  
LEFT JOIN orders o  
  ON c.customer_id = o.customer_id  
  AND o.order_status = 'Delivered';
```

Why this query?

If we put `o.order_status = 'Delivered'` in WHERE, customers with no delivered orders will disappear.

So the filter is placed inside ON.

13. Interview Questions and Answers

Q1. What is SQL logical execution order?

Answer:

```
FROM  
JOIN / ON  
WHERE  
GROUP BY  
HAVING  
SELECT  
DISTINCT  
ORDER BY  
LIMIT
```

Q2. Does SQL execute SELECT first?

Answer:

No.

Although SELECT is written first, SQL logically starts from FROM and JOIN.

Q3. Which executes first: JOIN or WHERE?

Answer:

JOIN executes first.

SQL first creates the joined dataset, then WHERE filters rows.

Q4. Which executes first: WHERE or GROUP BY?

Answer:

WHERE executes first.

Rows are filtered before grouping.

Q5. Which executes first: GROUP BY or HAVING?

Answer:

GROUP BY executes first.

HAVING filters the grouped result.

Q6. Why can't we use aggregate function in WHERE?

Answer:

Aggregate functions are calculated after GROUP BY.

WHERE runs before GROUP BY.

So aggregate functions like **SUM**, **COUNT**, and **AVG** cannot be used in WHERE.

Use HAVING instead.

Q7. Why can't we use SELECT alias in WHERE?

Answer:

WHERE executes before SELECT.

The alias is created in SELECT.

So WHERE does not know the alias yet.

Q8. Why can ORDER BY use SELECT alias?

Answer:

ORDER BY executes after SELECT.

So aliases created in SELECT are available to ORDER BY.

Q9. What is the difference between WHERE and HAVING?

Answer:

WHERE filters rows before grouping.

HAVING filters groups after grouping.

Example:

WHERE order_status = 'Delivered'

filters rows.

HAVING SUM(order_amount) > 50000

filters grouped results.

Q10. Why does LEFT JOIN sometimes behave like INNER JOIN?

Answer:

Because a WHERE condition on the right table removes NULL rows.

Example:

```
WHERE o.order_status = 'Delivered'
```

This removes customers with no orders.

To preserve all left table rows, put the filter in the ON condition.

Q11. Is logical execution order the same as physical execution order?

Answer:

No.

Logical execution order explains how SQL result is formed.

Physical execution order is decided by the database optimizer.

The optimizer can rearrange operations internally for performance, but the final result should follow SQL logic.

14. Final Revision Table

Clause	Purpose	Executes When
FROM	Select base table	1

JOIN / ON	Combine tables	2
WHERE	Filter rows	3
GROUP BY	Create groups	4
HAVING	Filter groups	5
SELECT	Choose final columns	6
DISTINCT	Remove duplicates	7
ORDER BY	Sort output	8
LIMIT	Restrict number of rows	9

15. One-Line Memory Trick

FROM data, JOIN tables, WHERE rows, GROUP them, HAVING filtered groups, SELECT columns, ORDER results, LIMIT output.

16. Final Takeaway

SQL execution order helps you debug queries faster.

Remember:

WHERE cannot use aggregates.

HAVING can use aggregates.

WHERE cannot use SELECT alias.

ORDER BY can use SELECT alias.

JOIN happens before WHERE.

GROUP BY happens before HAVING.

Once this order is clear, complex SQL queries become much easier to understand.